Лабораторная работа №10

Программирование в командном процессоре ОС UNIX

Выполнил: Мвикую Кристиан Кристофер
Группа: НБИБД-02-25
Дата: 29.04.2026
Проверил: _____

Содержание

- Цель работы
- Теоретическая часть
  - Командные процессоры (оболочки)
  - Переменные в языке программирования bash
  - Использование арифметических вычислений. Операторы let и read
  - Метасимволы и их экранирование
  - Командные файлы и функции
  - Передача параметров в командные файлы и специальные переменные
  - Управление последовательностью действий
- Ход работы
  - Задание 1: Резервное копирование самого себя
  - Задание 2: Обработка произвольного числа аргументов
  - Задание 3: Аналог команды ls
  - Задание 4: Подсчёт файлов по расширению
  - Дополнительные задания
- Ошибки и их решение
- Выводы
- Ответы на контрольные вопросы

Цель работы

Познакомиться с операционной системой Linux. Получить практические навыки работы с командным процессором bash и программированием в нём. Научиться создавать командные файлы (скрипты) для автоматизации задач администрирования и обработки данных.

Теоретическая часть

Командные процессоры (оболочки)

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. Командная оболочка принимает команды от пользователя, интерпретирует их и передаёт операционной системе для выполнения.

Основные реализации командных оболочек в UNIX/Linux:

Оболочка	Описание
sh (Bourne shell)	Стандартная командная оболочка UNIX/Linux, разработанная Стивеном Борном в 1977 году. Содержит базовый, но при этом полный набор функций. Является основой для многих других оболочек.
csh (C shell)	Разработана Биллом Джоем в 1978 году. Надстройка над оболочкой Борна, использующая С-подобный синтаксис команд с возможностью сохранения истории выполнения команд.
ksh (Korn shell)	Разработана Дэвидом Корном в 1983 году. Напоминает оболочку С, но операторы управления программой совместимы с операторами оболочки Борна.
bash (Bourne Again Shell)	Разработана Брайаном Фоксом для проекта GNU в 1989 году. В основе своей совмещает свойства оболочек С и Корна. Является стандартной оболочкой в большинстве дистрибутивов Linux.

POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна.

Переменные в языке программирования bash

Командный процессор bash обеспечивает возможность использования переменных типа **строка символов**. Имена переменных могут быть выбраны пользователем. Пользователь имеет возможность присвоить переменной значение некоторой строки символов.

Синтаксис переменных:

```
# Присваивание значения (без пробелов вокруг =)
mark=/usr/andy/bin

# Использование значения (с префиксом $)
echo $mark
echo ${mark}     # Безопасная форма с фигурными скобками

# Пример использования
mv afile ${mark}
```

Пояснение: Команда mv afile ${mark} переместит файл afile из текущего каталога в каталог с абсолютным полным именем /usr/andy/bin.

Массивы в bash:

```
# Создание массива с помощью команды set -A
set -A states Delaware Michigan "New Jersey"

# Добавление элемента по индексу
states[49]=Alaska

# Индексация массивов начинается с нулевого элемента
echo ${states[0]}    # Delaware
echo ${states[1]}    # Michigan
echo ${states[2]}    # New Jersey
echo ${states[49]}   # Alaska

# Получение всех элементов массива
echo ${states[@]}    # Все элементы
echo ${#states[@]}   # Количество элементов
```

Использование арифметических вычислений. Операторы let и read

Оболочка bash поддерживает встроенные арифметические функции.

Команда let:

Команда let является показателем того, что последующие аргументы представляют собой выражение, подлежащее вычислению.

```
# Простейшие арифметические операции
let sum=x+7          # Сложение
let diff=x-7         # Вычитание
let prod=x*7         # Умножение
let quot=x/7         # Целочисленное деление
let mod=x%7          # Остаток от деления

# Альтернативный синтаксис с двойными скобками
(( sum = x + 7 ))
(( sum++ ))          # Инкремент
(( sum-- ))          # Декремент
```

Особенность команды let: для идентификации переменной ей не нужен знак доллара. Можно писать let sum=x+7, и let будет искать переменную x.

Команда read:

Команда read позволяет читать значения переменных со стандартного ввода (клавиатуры).

```
echo "Please enter Month and Day of Birth ?"
read mon day trash
```

Пояснение: В переменные mon и day будут считаны соответственно первое и второе слова введённой строки. Переменная trash нужна для того, чтобы отобрать всю избыточно введённую информацию и игнорировать её.

Запись чисел в различных системах счисления:

```
# Формат: radix#number
let val=2#1010       # 10 в десятичной (двоичная)
let val=8#12         # 10 в десятичной (восьмеричная)
let val=16#A         # 10 в десятичной (шестнадцатеричная)
```

Метасимволы и их экранирование

При перечислении имён файлов текущего каталога можно использовать следующие символы:

Метасимвол	Значение	Пример
*	Соответствует произвольной, в том числе и пустой строке	echo * — выведет все файлы
?	Соответствует любому одинарному символу	ls prog.? — файлы prog.c, prog.o
	Соответствует	ls [a-z]* — файлы

[c1-c2]	любому символу между c1 и c2	на строчную букву

Экранирование метасимволов:

Метасимволы: < > * ? | \ & $ [ ]

Способ экранирования	Пример	Результат
Обратный слеш \	echo *	Выводит символ *
Одинарные кавычки '	echo 'ab*c'	Выводит строку ab*c
Двойные кавычки "	echo "$HOME"	Экранирует все, кроме $, \, `

Командные файлы и функции

Создание командного файла:

1. Создать текстовый файл с командами
2. Сделать файл исполняемым: chmod +x filename
3. Запустить: ./filename или bash filename

Функции в bash:

```
# Определение функции
function clist {
  cd $1
  ls
}

# Или альтернативный синтаксис
clist() {
  cd $1
  ls
}

# Вызов функции
clist /tmp

# Удаление функции
unset -f clist
```

Опции команды typeset для работы с функциями:

Опция	Назначение
-f	Перечисляет определённые на текущий момент функции При последующем вызове функции инициирует её

-ft	трассировку
-fx	Экспортирует функции в дочерние программы оболочек
-fu	Обозначает функции как автоматически загружаемые

Передача параметров в командные файлы и специальные переменные

Позиционные параметры:

При вызове командного файла параметры передаются через пробел. В командном файле к ним обращаются через $1, $2, ... $9.

```
# Пример скрипта where.sh
#!/bin/bash
who | grep $1
```

Вызов: ./where.sh andy — скрипт найдёт пользователя andy в выводе команды who.

Команда shift: удаляет первый параметр и сдвигает все остальные на места предыдущих.

Специальные переменные:

Переменная	Назначение	Пример
$0	Имя командного файла	./script.sh
$#	Количество параметров	3
$*	Все параметры как одна строка	"arg1 arg2 arg3"
$@	Все параметры как отдельные слова	"arg1" "arg2" "arg3"
$?	Код завершения последней команды	0 (успех)
$$	PID текущего процесса	12345
$!	PID последней фоновой команды	12346
$-	Значение флагов командного процессора	hB

Подстановки значений:

Синтаксис	Значение
${#name}	Длина строки в переменной name
${name:-value}	Если name не определён, подставить value
${name:=value}	Если name не определён, присвоить value
${name:?value}	Если name не определён, вывести ошибку
${name#pattern}	Удалить самый короткий левый образец

${name##pattern}	Удалить самый длинный левый образец
${name%pattern}	Удалить самый короткий правый образец
${name%%pattern}	Удалить самый длинный правый образец

Управление последовательностью действий

Оператор for:

```
for переменная in список
do
  команды
done
```

Пример:

```
for file in *
do
  echo "File: $file"
done
```

Оператор case:

```
case $переменная in
  шаблон1)
    команды1
    ;;
  шаблон2)
    команды2
    ;;
  *)
    команды_по_умолчанию
    ;;
esac
```

Условный оператор if:

```
if список-команд
then
  список-команд
elif список-команд
then
  список-команд
else
  список-команд
fi
```

Команда test используется для проверки условий:

Проверка	Описание
test -f файл	Файл существует и является обычным

test -d файл	Файл существует и является каталогом
test -r файл	Файл существует и доступен для чтения
test -w файл	Файл существует и доступен для записи
test -x файл	Файл существует и доступен для выполнения
test $a -eq $b	a равно b
test $a -ne $b	a не равно b
test $a -lt $b	a меньше b
test $a -gt $b	a больше b

Операторы цикла while и until:

```
# while — выполняется, пока условие истинно
while список-команд
do
  команды
done

# until — выполняется, пока условие ложно
until список-команд
do
  команды
done
```

Команды true и false:

- true — всегда возвращает код завершения 0 (истина)
- false — всегда возвращает код завершения 1 (ложь)

Используются для создания бесконечных циклов:

```
while true
do
  echo "hello"
  sleep 1
done
```

Прерывание циклов:

Команда	Действие
break	Полностью выходит из цикла
continue	Прерывает текущую итерацию и переходит к следующей

Ход работы

Создание каталога для лабораторной работы

```
cd ~/work/study/2025-2026/"Operating Systems"/os-intro/labs/
mkdir -p lab12
cd lab12
mkdir -p screenshots
```

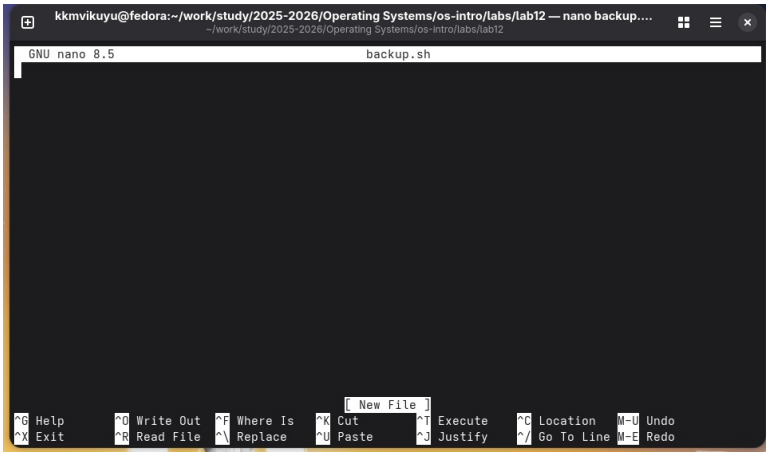
📸 **Screenshot:** Каталог lab12 успешно создан.

Задание 1: Резервное копирование самого себя

Требуется написать скрипт, который при запуске делает резервную копию самого себя (файла с исходным кодом) в директорию ~/backup с использованием архиватора tar.

Создание скрипта backup.sh

```
nano backup.sh
```



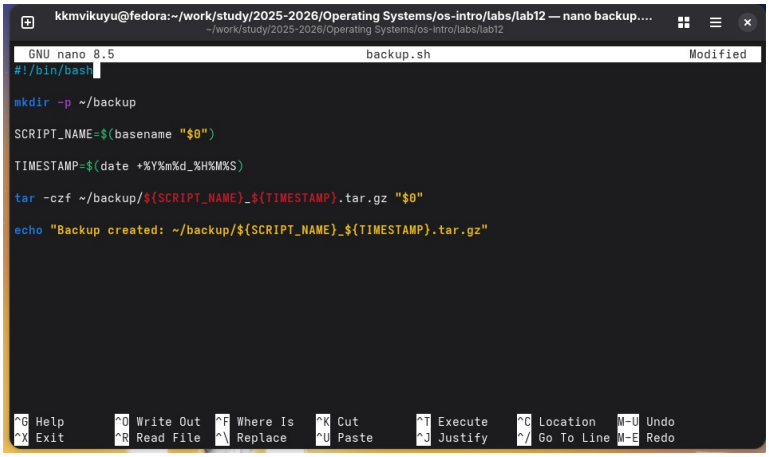
Создание скрипта backup.sh

Рисунок 1 - Создание файла backup.sh в редакторе nano

Исходный код скрипта backup.sh:

```
#!/bin/bash
# backup.sh - создаёт резервную копию самого себя

# Создание директории backup, если она не существует
mkdir -p ~/backup

# Получение имени скрипта без пути
SCRIPT_NAME=$(basename "$0")

# Создание временной метки (год-месяц-день_час-минута-секунда)
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Архивирование с помощью tar
# -c - создание архива
# -z - сжатие gzip
```

```
# -f - имя файла
tar -czf ~/backup/${SCRIPT_NAME}_${TIMESTAMP}.tar.gz "$0"

echo "Backup created: ~/backup/${SCRIPT_NAME}_${TIMESTAMP}.tar.gz"
```



Содержимое скрипта backup.sh

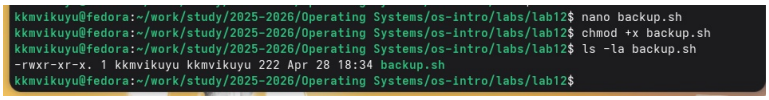
Рисунок 2 - Исходный код скрипта резервного копирования

Разбор ключевых моментов:

1. #!/bin/bash — шебанг, указывающий интерпретатор для выполнения скрипта
2. mkdir -p ~/backup — создаёт каталог ~/backup с родительскими каталогами (флаг -p)
3. $(basename "$0") — извлекает только имя файла из полного пути (удаляет путь)
4. $(date +%Y%m%d_%H%M%S) — генерирует строку с текущей датой и временем
5. tar -czf — ключи: c (create), z (gzip), f (file)

Установка прав и выполнение

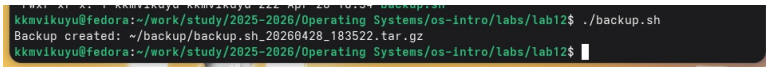
```
chmod +x backup.sh
ls -la backup.sh
./backup.sh
```



Установка прав на выполнение

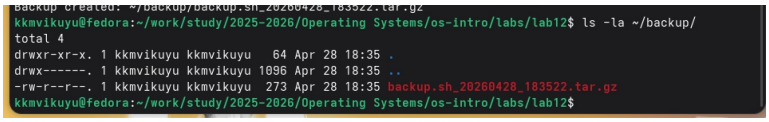
Рисунок 3 - Установка прав на выполнение (chmod +x) и проверка прав

Пояснение: Команда chmod +x добавляет право на выполнение. В выводе ls -la права -rwxr-xr-x означают: - r (read) — чтение - w (write) — запись - x (execute) — выполнение - Первая группа (rwx) — права владельца - Вторая группа (r-x) — права группы - Третья группа (r-x) — права остальных пользователей



Запуск backup.sh

Рисунок 4 - Результат выполнения скрипта backup.sh



Проверка созданной резервной копии

Рисунок 5 - Содержимое директории ~/backup с созданным архивом

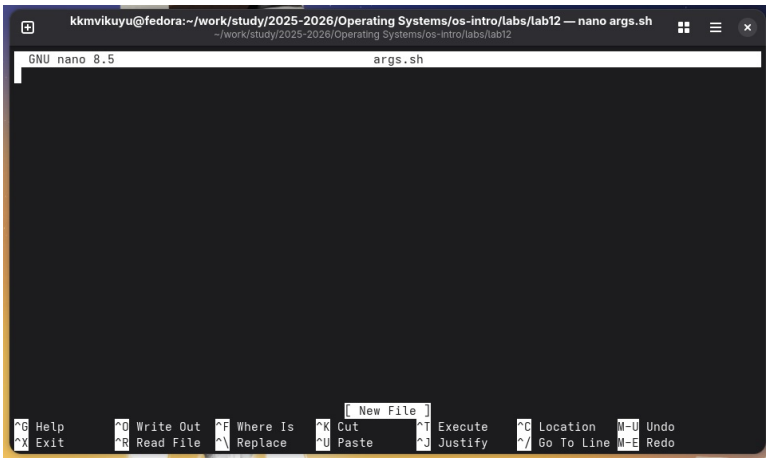
Пояснение: Скрипт успешно создал архив backup.sh_20260428_183522.tar.gz, содержащий исходный код скрипта.

Задание 2: Обработка произвольного числа аргументов

Требуется написать скрипт, обрабатывающий любое произвольное число аргументов командной строки (в том числе более десяти). Скрипт должен последовательно распечатывать значения всех переданных аргументов.

Создание скрипта args.sh

```
nano args.sh
```



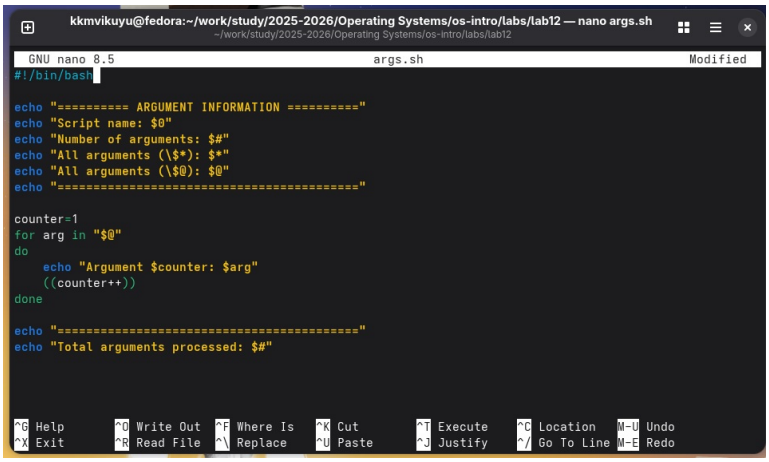
Создание скрипта args.sh

Рисунок 6 - Создание файла args.sh

Исходный код скрипта args.sh:

```
#!/bin/bash
# args.sh - выводит информацию о всех переданных аргументах

echo "========== ARGUMENT INFORMATION ==========="
echo "Script name: $0"
```

```
echo "Number of arguments: $#"
echo "All arguments (\$*): $*"
echo "All arguments (\$@): $@"
echo "========================================="

counter=1
for arg in "$@"
do
  echo "Argument $counter: $arg"
  ((counter++))
done

echo "========================================="
echo "Total arguments processed: $#"
```



Содержимое скрипта args.sh

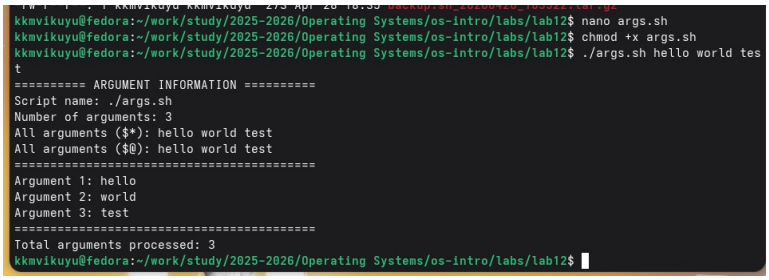
Рисунок 7 - Исходный код скрипта для обработки аргументов

Разбор ключевых моментов:

1. $0 — имя скрипта
2. $# — количество переданных аргументов
3. $* — все аргументы как одна строка
4. $@ — все аргументы как массив отдельных строк
5. Цикл for arg in "$@" — перебирает все аргументы по отдельности
6. ((counter++)) — инкремент счётчика

Тестирование скрипта

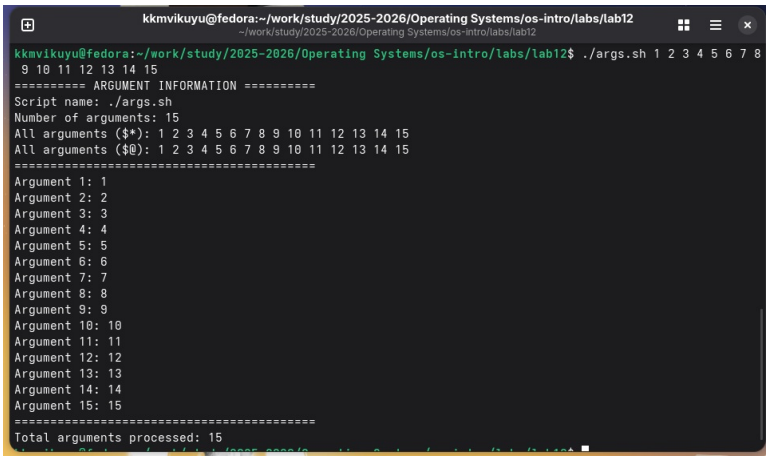
```
chmod +x args.sh
./args.sh hello world test
```



Тестирование args.sh с 3 аргументами

Рисунок 8 - Результат работы с тремя аргументами: hello, world, test

./args.sh 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15



Тестирование args.sh с 15 аргументами

Рисунок 9 - Результат работы с пятнадцатью аргументами

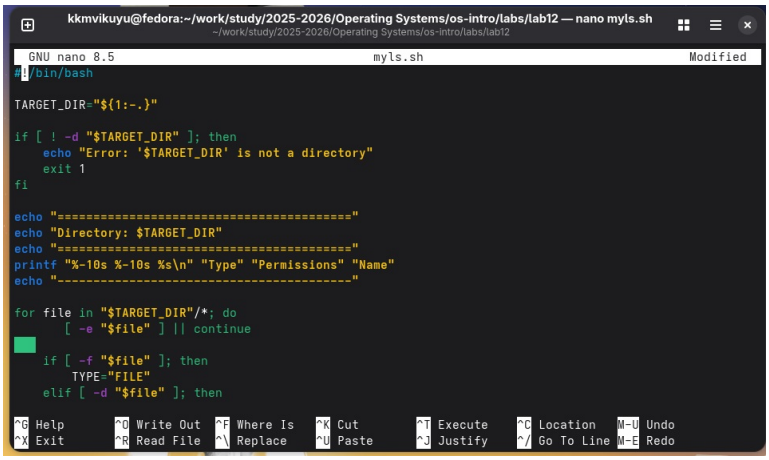
Пояснение: Скрипт успешно обработал 15 аргументов, что демонстрирует возможность работы с произвольным числом параметров, в том числе превышающим традиционное ограничение в 9 аргументов. Внутренний счётчик корректно отслеживает все аргументы.

Задание 3: Аналог команды ls

Требуется написать скрипт, который является аналогом команды ls (без использования самой этой команды и команды dir). Скрипт должен выдавать информацию о каталоге и выводить информацию о правах доступа к файлам.

Создание скрипта myls.sh

nano myls.sh



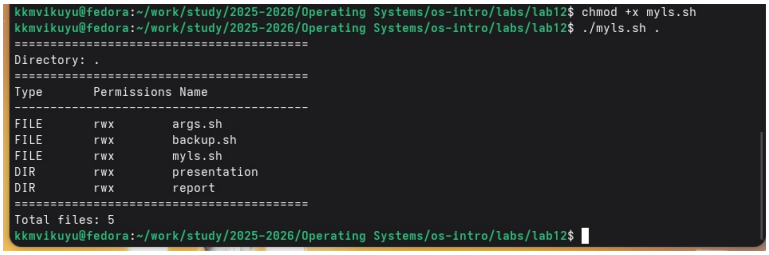
Создание скрипта myls.sh

Рисунок 10 - Создание файла myls.sh

Исходный код скрипта myls.sh:

```
#!/bin/bash
# myls.sh - аналог команды ls с информацией о типах файлов и правах

# Каталог для просмотра (по умолчанию текущий)
TARGET_DIR="${1:-.}"

# Проверка существования каталога
if [ ! -d "$TARGET_DIR" ]; then
  echo "Error: '$TARGET_DIR' is not a directory"
  exit 1
fi

echo "========================================"
echo "Directory: $TARGET_DIR"
echo "========================================"
printf "%-10s %-10s %s\n" "Type" "Permissions" "Name"
echo "----------------------------------------"

for file in "$TARGET_DIR"/*; do
  # Пропуск, если нет файлов
  [ -e "$file" ] || continue

  # Определение типа файла
  if [ -f "$file" ]; then
    TYPE="FILE"
  elif [ -d "$file" ]; then
    TYPE="DIR"
  elif [ -L "$file" ]; then
    TYPE="LINK"
  else
    TYPE="OTHER"
  fi
```

```
    # Формирование строки прав доступа
    PERMS=""
    [ -r "$file" ] && PERMS="${PERMS}r" || PERMS="${PERMS}-"
    [ -w "$file" ] && PERMS="${PERMS}w" || PERMS="${PERMS}-"
    [ -x "$file" ] && PERMS="${PERMS}x" || PERMS="${PERMS}-"

    # Получение имени файла без пути
    NAME=$(basename "$file")

    printf "%-10s %-10s %s\n" "$TYPE" "$PERMS" "$NAME"
done

echo "========================================="
echo "Total files: $(find "$TARGET_DIR" -maxdepth 1 -type f | wc -l)"
```



Содержимое скрипта myls.sh

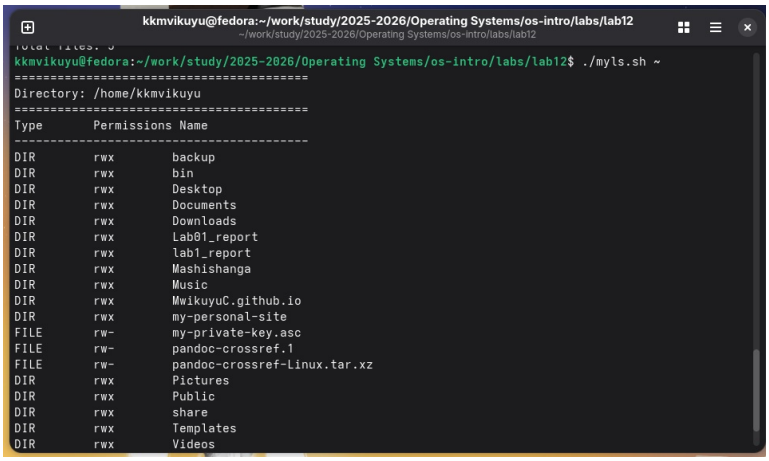
Рисунок 11 - Исходный код аналога команды ls

Разбор ключевых моментов:

1. "${1:-.}" — использует первый аргумент, если он есть, иначе текущий каталог (.)
2. [! -d "$TARGET_DIR"] — проверка, не является ли путь каталогом
3. printf "%-10s" — форматированный вывод с шириной 10 символов (выравнивание по левому краю)
4. [-e "$file"] — проверка существования файла
5. [-f "$file"] — проверка, является ли файл обычным
6. [-d "$file"] — проверка, является ли файл каталогом
7. [-L "$file"] — проверка, является ли файл символической ссылкой
8. [-r "$file"] — проверка права на чтение
9. [-w "$file"] — проверка права на запись
10. [-x "$file"] — проверка права на выполнение
11. $(basename "$file") — извлекает только имя файла

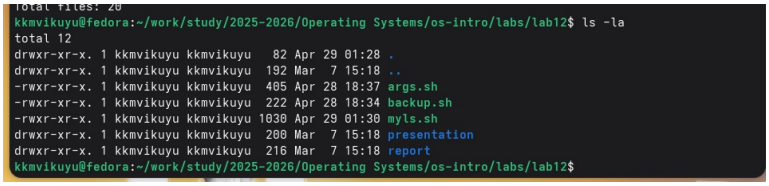
Выполнение скрипта myls.sh

```
chmod +x myls.sh
./myls.sh
./myls.sh ~
```



Выполнение myls.sh для домашнего каталога

Рисунок 12 - Результат работы myls.sh для каталога ~ (домашняя директория)



Сравнение с оригинальной командой ls

Рисунок 13 - Содержимое каталога с помощью команды ls -la для сравнения

Пояснение результатов:

Скрипт myls.sh выводит для каждого файла: - **Тип**: FILE (файл), DIR (каталог), LINK (ссылка) или OTHER (другое) - **Права доступа**: трёхсимвольная комбинация r (чтение), w (запись), x (выполнение) - **Имя**: имя файла без пути

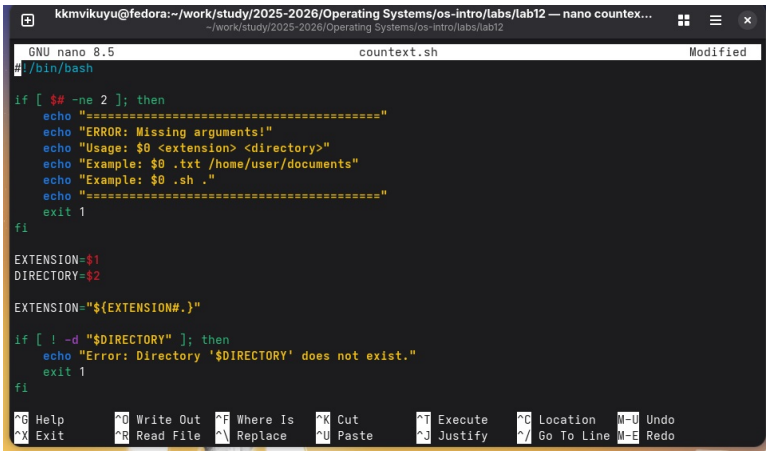
Сравнение с оригинальной командой ls -la показывает, что скрипт корректно определяет типы файлов и права доступа.

Задание 4: Подсчёт файлов по расширению

Требуется написать скрипт, который получает в качестве аргумента командной строки формат файла (.txt, .doc, .jpg, .pdf и т.д.) и вычисляет количество таких файлов в указанной директории.

Создание скрипта countext.sh

```
nano countext.sh
```



Создание скрипта countext.sh

Рисунок 14 - Создание файла countext.sh

Исходный код скрипта countext.sh:

```
#!/bin/bash
# countext.sh - подсчитывает количество файлов с заданным расширением

# Проверка наличия двух аргументов
if [ $# -ne 2 ]; then
   echo "========================================="
   echo "ERROR: Missing arguments!"
   echo "Usage: $0 <extension> <directory>"
   echo "Example: $0 .txt /home/user/documents"
   echo "Example: $0 .sh ."
   echo "========================================="
   exit 1
fi

EXTENSION=$1
DIRECTORY=$2

# Удаление точки из расширения, если она есть
EXTENSION="${EXTENSION#.}"

# Проверка существования директории
if [ ! -d "$DIRECTORY" ]; then
   echo "Error: Directory '$DIRECTORY' does not exist."
   exit 1
fi

echo "========================================="
echo "Counting *.$EXTENSION files in: $DIRECTORY"
echo "========================================="

# Подсчёт файлов с помощью find
```

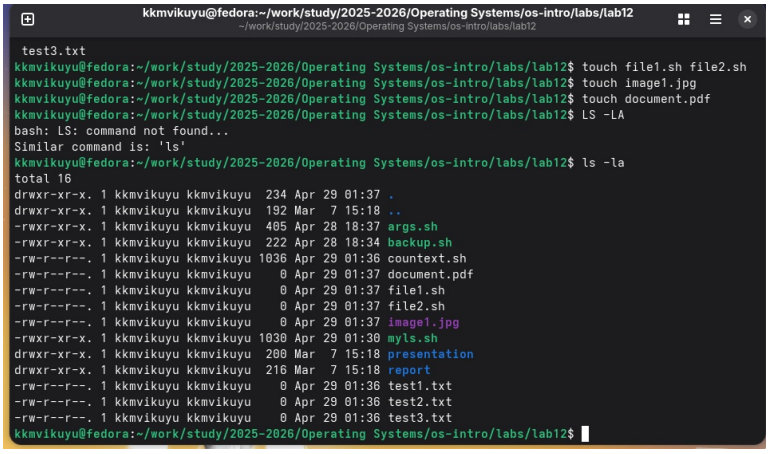
```
COUNT=$(find "$DIRECTORY" -maxdepth 1 -type f -name "*.$EXTENSION" 2>/dev/null |
            wc -l)

# Вывод списка найденных файлов
echo "Files found:"
find "$DIRECTORY" -maxdepth 1 -type f -name "*.$EXTENSION" 2>/dev/null | while
            read -r file; do
  echo "  - $(basename "$file")"
done

echo "========================================="
echo "TOTAL: $COUNT file(s) with extension .$EXTENSION"
echo "========================================="

# Возврат количества в качестве кода выхода
exit $COUNT
```

Разбор ключевых моментов:

1. [$# -ne 2] — проверка, что передано ровно 2 аргумента
2. EXTENSION="${EXTENSION#.}" — удаление ведущей точки из расширения
3. [! -d "$DIRECTORY"] — проверка существования директории
4. find "$DIRECTORY" -maxdepth 1 -type f -name "*.$EXTENSION" — поиск файлов:
   - -maxdepth 1 — только текущий уровень (без подкаталогов)
   - -type f — только обычные файлы
   - -name "*.$EXTENSION" — файлы с заданным расширением
5. 2>/dev/null — подавление сообщений об ошибках
6. wc -l — подсчёт количества строк
7. exit $COUNT — возврат количества в качестве кода выхода

Создание тестовых файлов

```
touch test1.txt test2.txt test3.txt
touch file1.sh file2.sh
touch image1.jpg
touch document.pdf
```

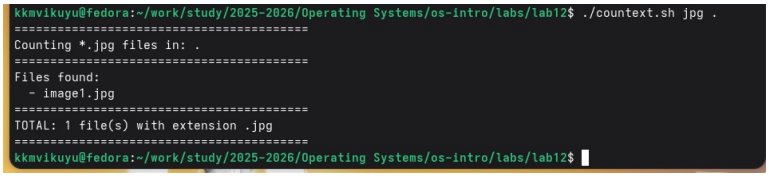


Создание тестовых файлов

Рисунок 15 - Создание файлов с различными расширениями для тестирования

Установка прав и выполнение

```
chmod +x countext.sh
./countext.sh txt .
./countext.sh sh .
./countext.sh jpg .
```

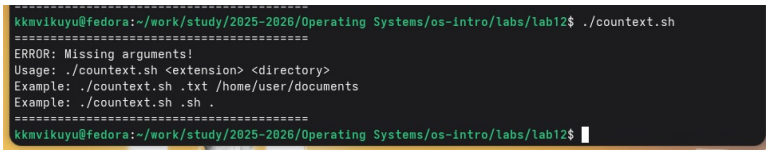


Подсчёт файлов .jpg

Рисунок 16 - Подсчёт файлов с расширением .jpg (найден 1 файл: image1.jpg)

Тестирование обработки ошибок

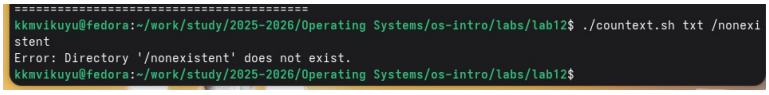
```
./countext.sh
```



Ошибка: отсутствие аргументов

Рисунок 17 - Сообщение об ошибке при отсутствии аргументов с инструкцией по использованию

```
./countext.sh txt /nonexistent
```



Ошибка: несуществующая директория

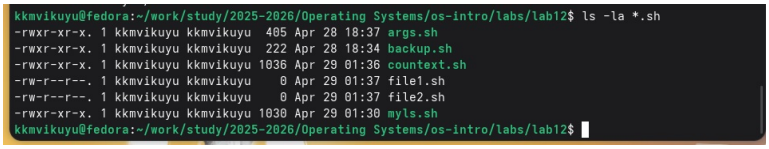
Рисунок 18 - Сообщение об ошибке при указании несуществующей директории

Пояснение: Скрипт корректно обрабатывает ошибочные ситуации, выводя понятные сообщения: - При отсутствии аргументов показывает справку с правильным синтаксисом - При указании несуществующей директории сообщает об ошибке

Дополнительные задания

Список всех созданных скриптов

```
ls -la *.sh
```

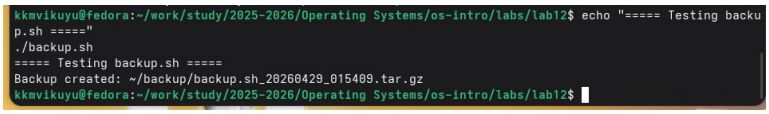


Список всех созданных скриптов

Рисунок 19 - Перечень всех исполняемых файлов в каталоге lab12

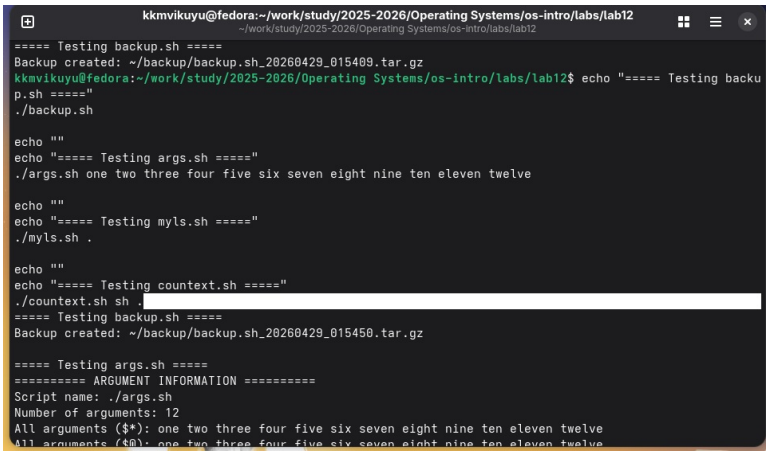
Комплексное тестирование всех скриптов

```
echo "===== Testing backup.sh ====="
./backup.sh

echo "===== Testing args.sh ====="
./args.sh one two three four five six seven eight nine ten eleven twelve

echo "===== Testing myls.sh ====="
./myls.sh .

echo "===== Testing countext.sh ====="
./countext.sh sh .
```



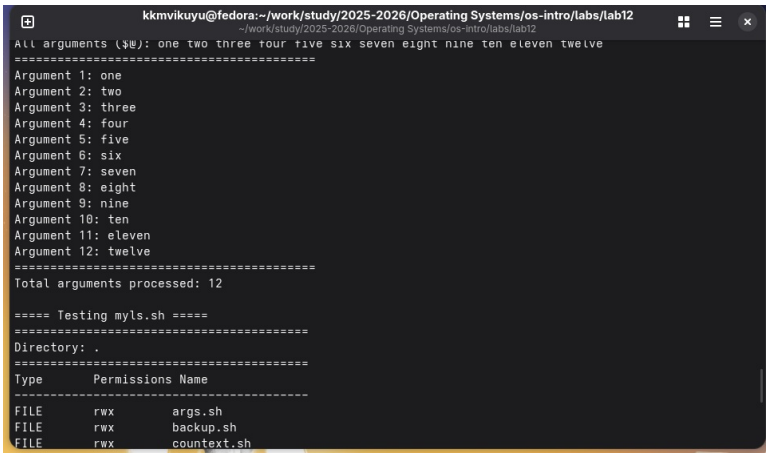
Запуск backup.sh отдельно

Рисунок 20 - Результат выполнения backup.sh



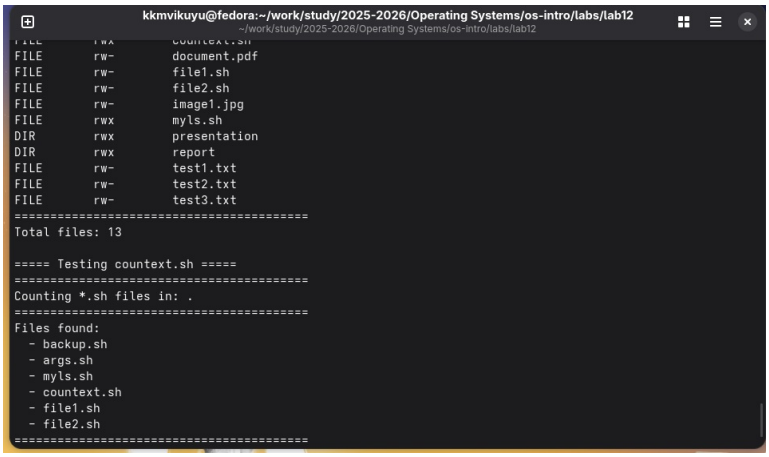
Запуск всех скриптов поочерёдно

Рисунок 21 - Последовательный запуск всех скриптов



Продолжение выполнения скриптов

Рисунок 22 - Результат работы args.sh с 12 аргументами и myls.sh

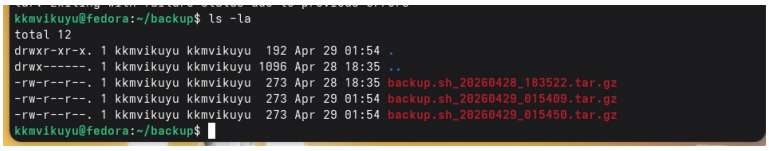


Результат работы countext.sh

Рисунок 23 - Результат подсчёта .sh файлов (найдено 6 файлов)

Проверка накопленных резервных копий

```
cd ~/backup
ls -la
```



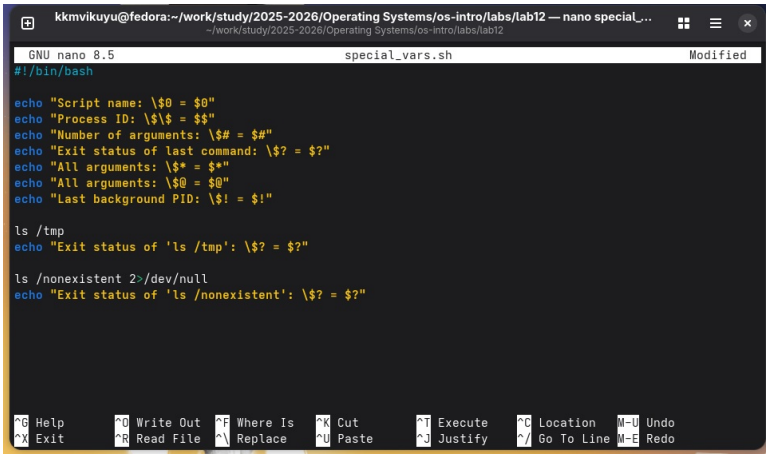
Содержимое ~/backup после нескольких запусков

Рисунок 24 - Все созданные резервные копии скриптов с разными временными метками

Специальные переменные bash

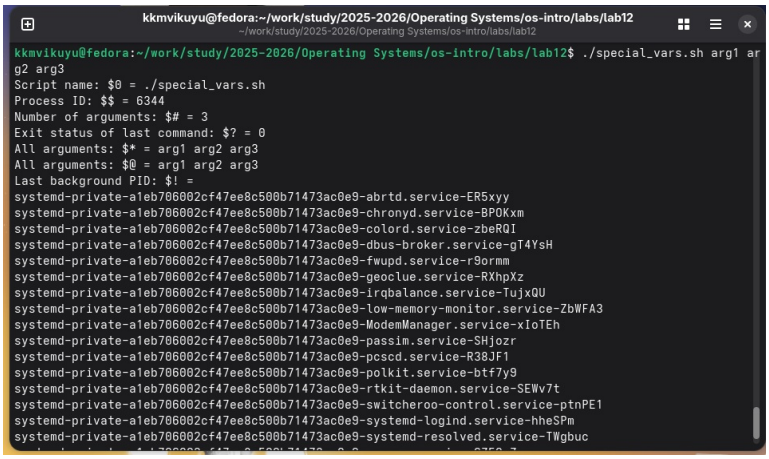
```
#!/bin/bash
# special_vars.sh - демонстрация специальных переменных
```

```
echo "Script name: \$0 = $0"
echo "Process ID: \$\$ = $$"
echo "Number of arguments: \$# = $#"
echo "Exit status of last command: \$? = $?"
echo "All arguments: \$* = $*"
echo "All arguments: \$@ = $@"
echo "Last background PID: \$! = $!"
```



Создание скрипта special_vars.sh

Рисунок 25 - Исходный код для демонстрации специальных переменных



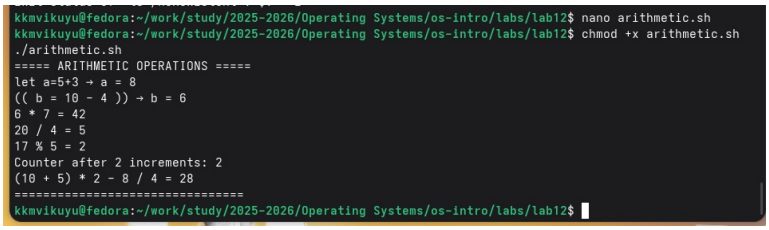
Выполнение special_vars.sh с аргументами

Рисунок 26 - Результат работы со специальными переменными

Арифметические операции

```
#!/bin/bash
# arithmetic.sh - демонстрация арифметических операций

echo "===== ARITHMETIC OPERATIONS ====="

let a=5+3
```

```
echo "let a=5+3 → a = $a"

(( b = 10 - 4 ))
echo "(( b = 10 - 4 )) → b = $b"

(( c = 6 * 7 ))
echo "6 * 7 = $c"

(( d = 20 / 4 ))
echo "20 / 4 = $d"

(( e = 17 % 5 ))
echo "17 % 5 = $e"

(( counter = 0 ))
(( counter++ ))
(( counter++ ))
echo "Counter after 2 increments: $counter"

(( result = (10 + 5) * 2 - 8 / 4 ))
echo "(10 + 5) * 2 - 8 / 4 = $result"
```



Арифметические операции

Рисунок 27 - Демонстрация арифметических операций в bash

Циклы и условные операторы

```bash #!/bin/bash # loops.sh - демонстрация циклов for, while, until и операторов break/continue

echo “===== FOR LOOP =====” for i in 1 2 3 4 5 do echo “Iteration: $i” done

echo “===== FOR LOOP WITH LIST =====” for file in *.sh do echo “Script file: $file” done

echo “===== WHILE LOOP =====” counter=1 while [ $counter -le 5 ] do echo “While loop count: $counter” ((counter++)) done

echo “===== UNTIL LOOP =====” counter=5 until [ $counter -lt 1
```